

Rapport d'Analyse Technique : Système de Simulation de Diffusion de Fumée

1 Fondements Théoriques et Dérivation des Formules

Pour simuler la diffusion de la fumée sur ordinateur, nous devons discrétiser les équations aux dérivées partielles continues.

1.1 Discrétisation de l'espace et indexation de la grille

Dans une grille bidimensionnelle, nous utilisons un tableau unidimensionnel pour stocker les états. La formule de correspondance d'index est la suivante :

$$idx(i, j, ny) = i \times ny + j$$

La concentration spatiale continue $q(x, y, t)$ est ainsi discrétisée en données de points de grille $q_{i,j}^n$.

1.2 Équation d'advection et schéma Upwind

L'advection décrit le processus par lequel la fumée est transportée par le vent.

Équation d'advection 1D :

$$\frac{\partial q}{\partial t} + u \frac{\partial q}{\partial x} = 0$$

Pour garantir la stabilité du calcul numérique, nous devons utiliser un **schéma Upwind** (décentré amont) :

- Lorsque $u > 0$: $\frac{q_i^{n+1} - q_i^n}{dt} + u \frac{q_i^n - q_{i-1}^n}{dx} = 0$
- Lorsque $u < 0$: $\frac{q_i^{n+1} - q_i^n}{dt} + u \frac{q_{i+1}^n - q_i^n}{dx} = 0$

Équation d'advection 2D :

$$\frac{\partial q}{\partial t} + u \frac{\partial q}{\partial x} + w \frac{\partial q}{\partial y} = 0$$

En définissant les paramètres sans dimension $a_x = \frac{|u|dt}{dx}$ et $a_y = \frac{|w|dt}{dy}$, l'équation de mise à jour explicite de l'advection 2D est :

$$q_{i,j}^{n+1} = (1 - a_x - a_y)q_{i,j}^n + a_x q_{up,x}^n + a_y q_{up,y}^n$$

Note : Nous utilisons une discrétisation temporelle explicite (Forward) car il est plus simple de calculer $q_{i,j}^{n+1}$ à partir des valeurs connues de $q_{i,j}^n$ (système causal), évitant ainsi la résolution de systèmes d'équations complexes propres aux schémas implicites.

1.3 Équation de diffusion

La diffusion décrit le processus de dispersion naturelle causé par le gradient de concentration. Son équation est :

$$\frac{\partial q}{\partial t} = \kappa \Delta q = \kappa \left(\frac{\partial^2 q}{\partial x^2} + \frac{\partial^2 q}{\partial y^2} \right)$$

En utilisant une différence centrale pour les dérivées secondes spatiales :

$$\frac{\partial^2 q}{\partial x^2} = \frac{q_{i+1,j}^n - 2q_{i,j}^n + q_{i-1,j}^n}{dx^2}$$

On obtient la formule finale de mise à jour de la diffusion :

$$q_{i,j}^{n+1} = q_{i,j}^n + \kappa dt \Delta q_{i,j}^n$$

1.4 Condition de stabilité CFL

Dans un pas de temps donné, la vitesse de propagation de l'information dans le modèle numérique doit être inférieure à la vitesse physique réelle, sous peine d'instabilité.

- **Limite d'advection** : $\frac{|u|dt}{dx} + \frac{|w|dt}{dy} < 1 \Rightarrow dt < \frac{1}{\frac{|u|}{dx} + \frac{|w|}{dy}}$. En pratique, un coefficient de sécurité est ajouté : $dt_a = \frac{0.8}{\frac{|u|}{dx} + \frac{|w|}{dy}}$.
- **Limite de diffusion** : $\frac{\kappa dt}{dx^2} + \frac{\kappa dt}{dy^2} < \frac{1}{2}$. En posant $h = \min(dx, dy)$, on obtient : $dt_d = \frac{\min(dx^2, dy^2)}{4\kappa}$.
- **Pas de temps final** : $dt = \min(dt_a, dt_d)$.

2 Analyse du Code et Architecture

Les dérivations physico-mathématiques précédentes sont implémentées à travers trois modules clés : `AdvectionSolver`, `SimulationController` et `MainWindow`.

2.1 `AdvectionSolver` : Le noyau de calcul numérique

C'est le moteur de calcul du système, dédié aux opérations sur tableaux.

- **Indexation et réduction de dimension** : Le code suit strictement la formule d'indexation via la fonction `inline int idx(int i, int j, int ny)`, permettant d'aplatir la grille 2D en un `std::vector` 1D pour un accès mémoire ultra-rapide.
- **Implémentation Upwind (`stepUpwind`)** : Cette fonction reproduit fidèlement la formule d'advection 2D. Elle utilise un opérateur ternaire pour déterminer la direction du vent : `const double q_up_x = (u >= 0.0) ? q[idx(i-1,j,ny)] : q[idx(i+1,j,ny)];`, lisant ainsi correctement la concentration "en amont".
- **Diffusion par différence centrale (`stepDiffusion`)** : Le code optimise les performances en pré-calculant l'inverse `invdx2 = 1.0 / (dx * dx)`, puis applique l'opérateur Laplacien pour mettre à jour la concentration.
- **Double buffering (Anti-corruption)** : Pour éviter que les nouvelles valeurs ne polluent les calculs de la même frame, des buffers temporaires (`tmp_` et `tmp2_`) sont utilisés. Une fois le calcul terminé, un `q.swap()` remplace globalement le champ de concentration.

2.2 `SimulationController` : Le cerveau et métronome du système

Ce module coordonne les flux de données et la progression temporelle.

- **Intégration des composants** : Il encapsule les données (`Smoke`), le solveur (`AdvectionSolver`), les champs de vent multiples (`std::vector<Wind>`) et le pilotage temporel (`QTimer`).
- **Initialisation (`setup`)** : Configure l'environnement au démarrage : grille de 220×220 , paramètres du vent par défaut et position de la cheminée.
- **Boucle principale (`tick`)** : Exécutée toutes les 20 ms, cette fonction applique rigoureusement la condition CFL de la Partie 1. Elle calcule le dt optimal, injecte la fumée continue, résout l'advection/diffusion, puis demande le rendu visuel.

2.3 `MainWindow` : Interface utilisateur et synchronisation

La fenêtre principale assure la synchronisation bidirectionnelle entre l'utilisateur et le moteur physique.

- **Gestion des vents (`syncControllerFromUi`)** : L'interface permet de combiner plusieurs sources de vent. Lorsqu'un curseur est déplacé, cette fonction met à jour les paramètres u et w du noyau de simulation en temps réel.
- **Rétroaction des résultats (`updateResultantLabel`)** : Pour valider l'effet cumulé des vents, la fenêtre interroge le contrôleur pour obtenir la vitesse et l'angle résultants (`resultantSpeed`, `resultantAngleDeg`) et les affiche instantanément, fermant ainsi la boucle de contrôle.